Computing (2022) 104:601–625
https://doi.org/10.1007/s00607-021-01030-9

REGULAR PAPER

Energy-efficient workflow scheduling with budget-deadline constraints for cloud

Ahmad Taghinezhad-Niar[1] · Saeid Pashazadeh[1] · Javid Taheri[2]

Received: 29 September 2020 / Accepted: 5 November 2021 / Published online: 13 January 2022
© The Author(s), under exclusive licence to Springer-Verlag GmbH Austria, part of Springer Nature 2021

Abstract

Cloud computing has become a well-known platform for solving big data and complex problems such as workflow applications. Infrastructure as a Service (IaaS) from the cloud is a suitable platform to solve these problems as it can potentially provide a nearly unlimited amount of resources using virtualization technology with a pay-per-use cost model. Various Quality of Service (QoS) objectives, such as cost and time, have been considered individually for workflow scheduling. In this paper, we proposed two energy-efficient heuristic algorithms with budget-deadline constraints that are appropriate for resources with Dynamic Voltage and Frequency Scaling (DVFS) enabled, as well as those that do not support DVFS. They are Budget Deadline Constrained Energy-aware (BDCE) and Budget Deadline DVFS-enabled energy-aware (BDD) algorithms for the cloud. Furthermore, they acquire affordable cost, faster scheduling length, and higher energy-saving ratio. Various evaluation metrics like success rate, cost and time ratios, energy consumption, utilization rate, and energy-saving ratio are utilized to evaluate the performance of the proposed algorithms. The obtained results are compared with budget-deadline constraints methods, such as BDSD, DBCS, and BDHEFT, as well as two other energy-efficient deadline-constrained algorithms, namely, ERES and Safari's algorithm in various scenarios on scientific workflow applications.

Keywords Workflow scheduling · Energy · Deadline · Budget · Cloud computing

Mathematics Subject Classification 68W05 · 90B36

✉ Saeid Pashazadeh
pashazadeh@tabrizu.ac.ir

Ahmad Taghinezhad-Niar
a.taghinezhad@tabrizu.ac.ir

Javid Taheri
javid.taheri@kau.se

1 Faculty of Electrical and Computer Engineering, University of Tabriz, Tabriz, Iran

2 Department of Computer Science, Karlstad University, Karlstad, Sweden

1 Introduction

High-performance computation is a requirement in today's life. Cloud computing is known as a suitable solution in this direction. A vast number and variety of applications migrated to the cloud for execution. These applications include a wide range of computation and data transferring time scoped in different areas [1]. These applications generate workloads with different types of tasks and precedence among them. An application constitutes a workflow that can be displayed by a Directed Acyclic Graph (DAG).

Two QoS metrics such as time and cost are considered important for cloud clients. Applications in disaster recovery and healthcare are examples of deadline-sensitive applications. Meanwhile, the monetary cost is considered a significant concern in pay-per-use services like the cloud [2]. Dramatic increases in cloud users and applications lead cloud providers to increase the capacity of their infrastructures, and that in turn causes a substantial increase in their energy consumption. Due to the increase of data center's energy consumption, the expense of cloud providers also increases. For example, an active data center with 1000 racks consumes 10 megawatts per day. Therefore, a data center with 50,000 servers spends 100 million kilowatts per hour in a year; this induces a financial cost of 52.3 million dollars for a year [3]. Besides, the high energy consumption of data centers induces an outstanding growth in the volume of carbon dioxide (CO_2) emission. Therefore, software-based methods to reduce energy consumption and CO_2 generating are notable [3]. The public cloud presents computing and storage services over the public internet. It offers on-demand services based on a pay-per-use price model. Most researchers developed workflow scheduling for a grid or cluster platform where the price model is different from cloud platforms. In cloud computing, pricing is based on resource leasing Billing Interval (BI). Therefore, clients must pay for an entire interval even if a fraction of that time is used. However, the monetary cost of a grid system is accounted for the accumulated price of used services. Additionally, the cloud supports on-demand services provisioned by a specific resource type based on a BI (usually hourly-based). In grid or cluster platforms, tasks are assigned to limited resources for execution. Most of the workflow scheduling algorithms are appropriate for grid platforms or heterogeneous resources [4–6]. Hence, workflow scheduling approaches in grid or cluster systems are not directly practical for clouds [7]. To address these issues, this paper proposes workflow scheduling for the public cloud that can provide on-demand services.

Scientific workflow scheduling is the main target of our algorithms. However, many other domains such as financial services, stock market analysis, weather forecasting, and back-end processing can also benefit our proposed solutions.

Leading cloud providers like Amazon Web Services (AWS) or Google provide utilities to manage a given budget. On the other hand, many deadline-constrained applications are classified into either Hard or Soft deadline constraint applications. Applications like disaster prediction that needs to notify natural disasters like a flood or hurricane are examples of such Hard deadline constraint applications where missing a deadline is not tolerable. Meanwhile, applications such as weather forecasting and medical tracking are examples of Soft deadline applications.

The scheduling length and monetary cost of the application are significant for cloud provider's clients. Meanwhile, cloud providers have to pay an enormous amount of money due to their energy consumption. To this end, the novel concept of green cloud computing is invented to address the extensive energy consumption of data centers and their generating carbon dioxides [8]. Therefore, the amount of energy consumption is crucial for cloud providers and the green environment perspective.

Various objectives are considered in scheduling solutions. In the case of a single objective, they are mostly scheduling length (makespan) in [9–11] or monetary cost in [7,12]. Mostly, user constraints like budget or deadline are individually considered in workflow scheduling [7,10]. Due to the NP-complete base of scheduling algorithms [13], considering multi-constraints scheduling algorithms over an infinite number of resources are significant. Several workflow scheduling algorithms consider simultaneous constraints like DBCS [4], BDSD [5], and BDHEFT [14]. The main objective of energy-efficient scheduling algorithms is executing applications with an energy-efficient approach so that their constraints are satisfied. Energy-aware scheduling algorithms mostly consider deadline constrained applications [1,6,15,16]. Monetary cost concerns a remarkable impact on the client's contentment. Therefore, energy consumption as an important QoS needs to be addressed alongside both user constraints (Deadline and Budget) which are the objectives of this paper.

BDSD and DBCS have proposed methods of workflow scheduling for heterogeneous resource environments assuming that there are arranged numbers of resources to execute tasks. However, in a public cloud environment like Amazon EC2, an unlimited number of available resources could be provisioned within a specific BI, which is usually hourly-based. Because none of these budget-deadline constrained algorithms are energy-aware, two recent energy-aware algorithms are utilized [15,16].

To this end, this paper proposes methods to facilitate three important QoS metrics (monetary cost, deadline and energy) in the cloud environment. All proposed methods are customizable based on the importance of quality metrics like time, cost, and energy efficiency. All applications that can be described as workflows and need to use cloud IaaS resources to perform their tasks can use the proposed algorithms.

On this set of problems, we proposed the Budget-Deadline Constrained energy-aware (BDCE) and Budget Deadline DVFS-enabled energy-aware (BDD) algorithms. These algorithms present a more eminent QoS to reduce the time and cost of the schedule plan and provide a greater utilization rate that can be customized for monetary cost or scheduling length preferences. Additionally, they propose an energy-efficient schedule plan to reduce the expenses of cloud providers. The proposed BDCE and BDD present a trade-off to balance three objectives that encompass cost, time, and energy consumption to address the proposed referenced problems. BDCE obtains a high success rate and achieves an affordable schedule plan within a green scheduling policy, and BDD is a DVFS-enabled energy-aware algorithm leading to a higher success rate and energy-efficient schedule plan.

Both algorithms are evaluated using multiple metrics: time and cost, energy consumption, energy-saving, utilization, and success rate with various constraints. Five algorithms are selected to compare the proposed algorithms. Two recent energy-aware scheduling algorithms were proposed by Neha et al. [15] and Safari et al. [16] under deadline constrained and three budget-deadline-constrained algorithms (BDSD [5],

DBCS [4] and BDHEFT [14]) are selected to evaluate our proposed algorithms. Table 1 shows the properties of all algorithms. Five synthetic scientific workflows are commonly used in this context to evaluate all algorithms.

This paper's contributions are as the following:

- Two energy-aware workflow scheduling algorithms (BDCE and BDD) that can simultaneously handle budget and deadline constraints;
- Extensive evaluation of several workflow scheduling methods and comparing them based on their time, monetary cost, success rate, energy-saving ratio, and utilization rate in various constraints;

Our paper is organized as the following. Section 2 demonstrates the related works. The problem is described in Sect. 3. The proposed work is described in sect. 4. Section 5 illustrates the evaluation properties. Section 6 articulates the experimental results. Section 7 contains the conclusion and future works.

2 Related works

Scheduling problems can be classified into two groups: those that are designed for independent tasks and those that are designed for workflows. Independent tasks can be run in any order, whereas data dependencies exist among tasks in a workflow. Therefore, there are precedence dependencies among tasks. In this paper, we study the scheduling of workflow tasks.

Meta-heuristic algorithms are utilized for workflow scheduling problems like Genetic Algorithm and Particle swarm Optimization [17,18]. Meta-heuristic algorithms may require an unacceptable amount of time to solve real cases [4], and therefore heuristic algorithms are more common to schedule workflow applications.

In heuristic algorithms, most researchers study deadline-constrained [15,16] or budget-constrained algorithms [19]. Several studies considered both budget and deadline simultaneously [4,5,14].

Sun et al. [5] proposed the BDSD algorithm for addressing budget-deadline constraints. The BDSD presents an approach based on critical paths. It calculates a deadline for individual tasks based on the longest time to execute all tasks in the critical path. They used a grid-based price model with limited resources.

Verma et al. [14] proposed a cost and time efficient algorithm named BDHEFT. The BDHEFT is a workflow scheduling algorithm for the cloud based on the HEFT [11] algorithm. For both the whole workflow and individual tasks, BDHEFT provides a spare budget and deadline. As a result, it generates a balancing value to determine the best resource allocation for each task. They used two evaluation metrics: normalized cost and normalized scheduling length.

A budget-deadline constrained scheduling for heterogeneous resources named DBCS is presented in [4]. Their algorithm is based on HEFT [11]. They presented a balance factor to investigate the candidate resources for each task. Their algorithm is feasible for a heterogeneous resource environment or grid. None of these three budget-deadline algorithms are energy-aware.

Energy-aware algorithms are considered mostly with a deadline. Stavrinides et al. [20] proposed a deadline-constrained energy-aware algorithm for workflow applications. They utilized the DVFS technique for fitting tasks into the schedule gap spaces created from workflow dependencies. Metrics like monetary cost, energy consumption, and success rate in deadlines are evaluated in their study.

Li et al. [21] proposed a deadline-constrained heuristic algorithm to minimize the cost and energy of the scientific workflows in the cloud environment named CEAS. Their proposed algorithm contains five sub-algorithms that lead to the high time complexity of $O(n^4)$.

Tang et al. [22] presented a HEFT [11] base energy-efficient approach. Their algorithm first acquires a HEFT schedule plan. Second, they used a heuristic algorithm, to change the task's resources to lower energy-consuming resources without exceeding the deadline. Third, each resource frequency is lowered based on the DVFS technique if it does not violate the deadline.

Safari et al. [16] proposed a deadline-constrained energy-aware algorithm based on critical paths suitable for clouds. They categorized workflow tasks into different critical paths. Then, each critical path is assigned to a resource with the minimum energy consumption only if it meets the deadline. They also proposed a DVFS-enabled algorithm to reduce the energy consumption of resources. This energy-aware deadline-constrained algorithm is selected to compare with our proposed algorithms. Safari's algorithm does not consider cost as an objective, and therefore its lower energy consumption usually leads to higher monetary costs.

Garg et al. [15] presented the ERES algorithm that is a heuristic energy-efficient scheduling method for clouds. They presented methods for obtaining an energy-efficient schedule plan, like allocating successor tasks to the resource that executed their predecessors. Their algorithm is a deadline-constrained algorithm aiming to reduce cost by considering resource utilization rate along with energy to reduce the energy consumption of the schedule plan. The ERES algorithm is also compared with our proposed algorithms. When compared with energy-aware algorithms such as Safari's algorithm, its lower resource cost results in poorer energy performance. It is because they use preliminary techniques such as assigning two dependent tasks to a resource without considering BIs.

The Public Cloud offers elastic and unlimited resources. These resources are billed based on a BI that is usually considered hourly, like Amazon EC2. However, algorithms like BDSD and DBCS that consider limited resources without considering BIs are not suitable for cloud services with elastic and unlimited resources.

To the best of our knowledge, our presented algorithms are the first heuristic workflow scheduling algorithms that consider the budget and deadline constraints while being energy efficient. Hence, we used two recent energy-aware deadline-constrained algorithms to evaluate the energy effectiveness of all proposed algorithms. Table 1 shows the features of all algorithms.

Table 1 Properties of workflow scheduling algorithms

Algorithms	Deadline	Budget	Energy-awareness	Environment
BDSD [5]	*	*		Heterogeneous resources
BDHEFT [14]	*	*		Cloud
DBCS [4]	*	*		Heterogeneous resources
ERES [15]	*		*	Cloud
Safari et al. [16]	*		*	Cloud
BDEC	*	*	*	Cloud
BDD	*	*	*	Cloud

3 Problem description

3.1 Cloud modeling

The proposed methods are presented based on the cloud IaaS. In this service, particular resources with various configurations in CPU, memory, storage, and bandwidth with different pricing are available based on a BI. BI is based on different periods on various cloud services. We considered on-demanded resource types with hourly-based BI for performance evaluation purposes. Let assume $RT = \{rt_1, rt_2, \ldots, rt_m\}$ represents different configurations with various provisioning prices for each BI. $r_{i,k}$ represents a provisioned resource from RT where i means id of the resource and k refers to the resource type configuration. A different number of instances from each configuration of RT can be provisioned for a demanded number of BIs. Each resource CPU supports a DVFS technology. Each CPU can run on different performance states (P-State) [1]. These P-States are denoted like $(f_0^{r_j}, V_0^{r_j}), (f_1^{r_j}, V_1^{r_j}), \ldots, (f_z^{r_j}, V_z^{r_j})$. Hence, PS_0 indicates the voltage and frequency set $(f_0^{r_j}, V_0^{r_j})$ which is the highest level of frequency in the resource.

3.2 Workflow model

Workflow is demonstrated by a DAG. $W = (T, R)$ indicates a workflow with its tasks and tasks' relationships. $T = \{t_0, t_1, t_2, \ldots, t_n\}$ represent tasks of a workflow. $R = \{r_{(i,j)} | (t_i, t_j \in T)\}$ is a set of relationships that illustrates tasks' precedence. For example, $r_{(i,j)}$ represents that t_i is a direct precedence of t_j, i.e. t_j consumes the data coming from t_i. Several tasks can be predecessors of a task; that is, t_i can be considered for execution, only if all its precedents are already executed, and t_i's input data is available. A task t_i direct predecessors are defined: $prd(t_i) = \{t_p | (t_p, t_i) \in T\}$ and its direct successors are defined as $suc(t_i) = \{t_s | (t_i, t_s) \in T\}$. The execution time of task t_i in resource r_j in frequency level (f_s) is denoted in Eq. (1), where $w(t_i, r_j)$ denotes the execution time of task t_i on resource r_j. The DVFS technology [23,24] reduces the processor's voltage and frequency set to conserve energy. The resource frequency level with the highest frequency and voltage set (PS_0) is 100% considering

the linear connection between voltage and frequency set of the resource processor [1,16].

$$EX_{t_i}^{r_j, f_s} = \frac{w(t_i, r_j)}{f_s} \tag{1}$$

Transfer time for sending files of t_i to t_j is defined in Eq. (2). When two tasks are executed in the same IaaS resource, the data transfer time between them becomes negligible and thus is considered zero for simplification. r_{t_i} is the IaaS resource that executed the task t_i. DS_{t_i,t_j} indicates the total data size of the files that must be transferred from the r_{t_i} to r_{t_j}.

$$TF_{t_i,t_j} = \begin{cases} \frac{|DS_{t_i,t_j}|}{bw}, & r_{t_i} \neq r_{t_j} \\ 0, & r_{t_i} = r_{t_j}. \end{cases} \tag{2}$$

Equation (3) defines the start time of task t_i in resource r_j, where τ denotes the system time at the moment of scheduling the task. AV_{r_j} denotes the resource r_j available time.

$$ST_{t_i}^{r_j} = \max\left\{\tau, AV_{r_j}, \max_{t_p \in prd(t_i)} \left\{FT_{t_p} + TF_{t_p,t_i}\right\}\right\} \tag{3}$$

To determine a task's estimated finish time on each resource, the notion FT in Eq. (4) is defined.

$$FT_{t_i}^{r_j, f_s} = ST_{t_i}^{r_j} + EX_{t_i}^{r_j, f_s} \tag{4}$$

3.3 Energy model

In a cloud data center, most energy consumption is caused by the processors, memory, storage, and network devices. The processor alone consumes between 37% and 43% of the total energy [15]. Nevertheless, because the costs related to memory, storage, and network devices are almost fixed and do not change based on executing any specific workflow, in this paper, we only consider the energy consumption of processors that is greatly related to the execution of a workflow. The energy consumption of a resource is calculated by the accumulation of processors' static and dynamic energy. When a resource is provisioned, the static energy is constant, however, dynamic energy depends on the frequency and voltage of the processor. We assumed zero overhead for the frequency transition which is a very small amount of time (in the range of microseconds) compared to others. The total energy consumption of a processor belongs to the dynamic energy consumption [22]. In this paper, we consider dynamic energy consumption defined in Eq. (5). The expressions V, f, and C indicate the voltage, frequency, and the constant related to the processor's capacity, respectively. Consequently, energy consumption is classified into two parts: when a resource is idle and when it is busy. When a resource is idle, its processor's frequency and voltage

are considered at their minimum levels. The total energy consumption of a resource is defined in Eq. (6).

$$Energy_{dynamic} = C.V^2.f \tag{5}$$

$$Energy_{r_j}^{total} = Energy_{r_j}^{idle} + Energy_{r_j}^{busy} \tag{6}$$

When resource r_j is busy (i.e., executing a task), its energy consumption is calculated by Eq. (7) where V_{r_j,t_i} and f_{r_j,t_i} indicate the voltage and frequency of resource r_j executing task t_i, respectively.

$$Energy_{r_j}^{busy} = \sum_{t_i \in r_j} C \cdot V_{r_j,t_i}^2 \cdot f_{r_j,t_i} \cdot EX_{t_i}^{r_j,f_s} \tag{7}$$

When a resource is idle, its processor works at the minimum frequency and voltage levels. Energy consumption of a resource during its idle time is defined in Eq. (8) where $idle_{r_j}$ indicates the idle time of r_j.

$$Energy_{r_j}^{idle} = C \cdot V_{\min}^2 \cdot f_{\min} \cdot idle_{r_j} \tag{8}$$

3.4 Problem definition

Tasks' execution time is assumed to be known, as most of the workflow scheduling studies [5,14,15]. This paper utilizes synthetic workflows that are generally used to evaluate workflow scheduling methods [5,15]. These synthetic workflows are produced by tracing scientific applications [25]. These workflow applications contain various DAG structures and different computation and communication attributes. Two dummy tasks (entry and exit) are added to each DAG. Workflow's scheduling length (SL) is defined in Eq. (9).

$$\mathrm{SL} = FT(t_{exit}) \tag{9}$$

Completion time (CP) for a task is defined in Eq. (10) where $EX_{t_i}^{\min}$ indicates the minimum execution time of t_i on the fastest resource. The highest performance resource is utilized to acquire the Earliest Start Time (EST) in Eq. (11).

$$CP(t_i) = EX_{t_i}^{\min} + \max_{t_p \in prd(t_i)} \left\{ TF_{t_p,t_i} \right\} \tag{10}$$

$$EST_{t_i} = \begin{cases} 0, & t_i = t_{entry} \\ \max\limits_{t_p \in prd(t_i)} \left\{ EST_{t_p} + CP(t_p) \right\}, & otherwise \end{cases} \tag{11}$$

Equation (12) calculates the number of BIs that resource r_j needs. Hourly-based BI is considered in this paper. The energy consumption of a task in a resource is calculated

by Eq. (13).

$$BC_{r_j} = \left\lceil \frac{FT(r_j)}{\mathrm{BI}} \right\rceil \tag{12}$$

$$EC_{t_i}^{r_j, f_s} = C.V^2 \cdot f \cdot EX_{t_i}^{r_j, f_s} \tag{13}$$

$FT(r_j)$ denotes r_j's finish time. The monetary cost of a task for execution on a resource is defined in Eq. (14). $FT_{t_i}^{r_j}$ denotes the finish time of t_i on resource r_j. $Price_{r_j}$ denotes the price of r_j for a BI.

$$TC_{t_i}^{r_j, f_s} = \begin{cases} \left\lceil \frac{FT_{t_i}^{r_j, f_s}}{\mathrm{BI}} \right\rceil \cdot Price_{r_j}, & otherwise \\ 0, & \left\lceil \frac{FT_{t_i}^{r_j, f_s}}{\mathrm{BI}} \right\rceil \leq BC_{r_j} \end{cases} \tag{14}$$

If a task can execute on the resource r_j before BC_{r_j}, then it takes zero cost for executing on that resource. To calculate the total monetary cost of a workflow, the notion WF_{cost} in Eq. (15) is defined.

$$WF_{cost} = \sum_{i=1}^{n} TC_{t_i} \tag{15}$$

4 Proposed algorithms

This section illustrates two heuristic energy-aware algorithms with simultaneous budget and deadline constraints, named BDCE and BDD. The proposed algorithms categorize tasks into levels. BDCE calculates and then distributes a given budget and deadline for each level. BDD considers the remaining budget for the deadline of each task. Both algorithms select tasks based on their priority before starting the resource provisioning step. Resource provisioning examines resources to find the one that satisfies all objectives (energy, cost, and time).

4.1 Categorizing tasks into levels

In this step, a method based on the DBL algorithm [26] is utilized to categorize tasks into different levels so that the tasks in each level do not have any precedent relationship. id of each level is obtained by Eq. (16). DP_{t_i} denotes the distance of t_i from the exit task. L_k in Eq. (17) represents the group of tasks that have identical DP values.

$$DP_{t_i} = \begin{cases} \max\limits_{t_s \in suc(t_i)} \{DP_{t_s} + 1\}, & otherwise \\ 0, & t_i = t_{exit}. \end{cases} \tag{16}$$

$$L_k = \{t_i | DP_{t_i} = k\} \tag{17}$$

4.2 Deadline distribution

In this step, the deadline will be distributed into the levels described in Sect. 4.1. First, σ is calculated by Eq. (18) as the sum of workflow tasks' completion time.

$$\sigma = \sum_{i=1}^{n} CP(t_i) \tag{18}$$

Then, each level completion time (LCP_k) is calculated by Eq. (19) as the sum of level k tasks' completion time.

$$LCP_k = \sum_{t_i \in L_k} CP(t_i) \tag{19}$$

Then a deadline (DL) for each level is obtained by Eq. (20). GDL denotes the total deadline. Algorithm 1 elaborates the deadline distribution, starting from the level containing the entry task.

$$DL_k = DL_{k+1} + \left(LCP_k \cdot \frac{GDL}{\sigma} \right) \tag{20}$$

Algorithm 1: Deadline-Distribution

```
Calculate σ by Eq.(18) ;
Calculate LCP_k by Eq.(19) ;
foreach L_k : levels do
    Calculate DL_k by Eq. (20);
end
```

4.3 Budget distribution

This step calculates each level's budget share from the given budget. Eq. (22) denotes the budget distribution of the BDCE algorithm into levels. BL_k means the dedicated budget for L_k. The Remaining Budget (RMB) is calculated by Eq. (21), where GBD denotes the user given budget and CBD denotes the current consumed budget (up to the current scheduling process). The BDD algorithm uses Eq. (21) as task's BL and BDCE algorithm calculates BL based on Eq. (22) illustrated in Algorithm 2.BL_{k+1} denotes the remaining budget of the level (dedicated budget − consumed budget) from level L_{k+1}. Therefore, it will only be used in resource provisioning to obtain a budget

constraint for the task. Hence, during the resource provisioning, the budget constraint of tasks is acquired from BL and RMB for BDCE and BDD algorithms, respectively.

$$RMB = GBD - CBD \tag{21}$$

$$BL_k = BL_{k+1} + \left(LCP_k \cdot \frac{GBD}{\sigma} \right) \tag{22}$$

Algorithm 2: Budget-Distribution of BDCE algorithm

```
Calculate σ by Eq.(18) ;
foreach L_k : levels do
    Calculate LCP_k from L_k's tasks by Eq. (19) ;
    Calculate BL_k by Eq. (22);
end
```

4.4 Task selection

In this step, tasks are sent for execution in the resource provisioning step. If all direct predecessors of a task are executed, it will be considered a ready task. Tasks are selected by the order of their EST from ready tasks.

4.5 Resource provisioning

This step selects a candidate resource for the execution of a task. If no provisioned resource is available to execute the task by its constraints, it then provisions a new resource from the offered resource types to execute the task. This step investigates resources for a solution that satisfies the application's budget-deadline constraints and provides an energy-efficient schedule plan. Before starting to schedule, the provisioned resources list is empty. Therefore, a resource must be provisioned from resource types. Three formulas are proposed to investigate the task's candidate resources. $\phi Time$ defined in Eq. (23) rates each resource based on the task's execution time. The maximum value returned from Eq. (23) is the one that belongs to the fastest resource. $\phi Cost$ defined in Eq. (24) rates resources based on the task's monetary cost for execution on them. The maximum value of $\phi Cost$ is the most affordable resource for the execution of the task. $\phi Energy$ defined in Eq. (25) rates resources based on their energy consumption. It rates the most energy-efficient resource with '1'. When a constraint is violated, the related statement (i.e., $\phi Cost$ or $\phi Time$) will return a negative value. However, $\phi Energy$ returns zero for the worst energy-efficient resource. Therefore, when a deadline or budget is missed, it causes a bigger impact on Eq. (29) than the energy. $FT_{t_i}^{\min}$ denotes the finish time of task t_i in the fastest resource. $TC_{t_i}^{\min}$ indicates

the cost of the most affordable resource for executing t_i.

$$\phi Time_{t_i}^{r_j,f_s} = \frac{DL_k - \left(FT_{t_i}^{r_j,f_s} - FT_{t_i}^{\min}\right)}{DL_k}, t_i \in L_k \tag{23}$$

$$\phi Cost_{t_i}^{r_j,f_s} = \frac{BL_k - \left(TC_{t_i}^{r_j,f_s} - TC_{t_i}^{\min}\right)}{BL_k}, t_i \in L_k \tag{24}$$

$$\phi Energy_{t_i}^{r_j,f_s} = \frac{EC_{t_i}^{\max} - EC_{t_i}^{r_j,f_s}}{EC_{t_i}^{\max} - EC_{t_i}^{\min}}, t_i \in L_k \tag{25}$$

Subsequently, a trade-off statement for balancing QoS parameters such as time, cost, and energy is defined by Eq. (29). λ is a coefficient from [0, 1] to manipulate budget and deadline constraints' impact on energy. ω is used to control essentially of the time to cost.

The slack time (S) of task t_i on resource r_j is calculated by Eq. (26). The minimum frequency of resource r_j that is needed to execute t_i before each task's deadline is calculated by Eq. (27). Eq. (28) defines the minimum frequency that is suitable to execute task t_i by the resource r_j. For example, if Eq. (28) results in a lower frequency than the minimum supported frequency of the resource r_j, then the minimum frequency of the resource is selected. The frequency level of the frequency acquired from Eq. (28) is used in Eq. (29).

$$S_{t_i}^{r_j} = \frac{FT_{t_i}^{r_j}}{DL_{t_i}} \tag{26}$$

$$minf_{t_i}^{r_j} = f_{\max}^{r_j} \cdot S_{t_i}^{r_j} \tag{27}$$

$$f_{t_i}^{r_j} = \begin{cases} f_{\min}^{r_j}, & otherwise \\ minf_{t_i}^{r_j}, & minf_{t_i}^{r_j} >= f_{\min}^{r_j} \end{cases} \tag{28}$$

In Algorithm 3, ϕ_{t_i,r_j} is investigated on provisioned resources and resource types. If $max\phi$ belongs to resource types, it provides a resource from the resource type and adds it to the list of resources. Afterward, the task t_i is assigned to the selected resource. The resource provisioning step is illustrated by Algorithm 3; it is followed by Algorithm 4 to calculate each resource's ϕ value.

Algorithm 4 investigates the ϕ value defined in Eq. (29). It calculates the minimum frequency of the resource that does not violate the deadline (Lines 3–7). For non-DVFS enabled resources, BDCE calculates ϕ with the maximum frequency of r_j, without Lines 3–7 in Algorithm 4.

$$\phi_{t_i}^{r_j,f_s} = \lambda \cdot \left(\omega \cdot \phi Time_{t_i}^{r_j,f_s} + (1-\omega) \cdot \phi Cost_{t_i}^{r_j,f_s}\right) + \left((1-\lambda) \cdot \phi Energy_{t_i}^{r_j,f_s}\right) \tag{29}$$

Algorithm 3: Resource-Provisioning

Input: t_i
$max\phi \leftarrow -\infty$;;
foreach r_j : $resources$ **do**
  $max\phi \leftarrow$ Quality-Calculation($t_i, r_j, max\phi$);
end
foreach rt_k : $resourceTypes$ **do**
  $max\phi \leftarrow$Quality-Calculation($t_i, rt_k, max\phi$);
end
if *$max\phi$ belongs to a resourcesType* **then**
  provision a new resource from the resource type ;
  $resources \leftarrow resources \cup$ { the resource type};
end
t_i is allocated to the resource with $max\phi$;
t_i is removed from ready tasks;

Algorithm 4: Quality-Calculation

Input: $t_i, r_j, max\phi$
calculate $\phi_{t_i}^{r_j}$ by Eq. (29);
if $max\phi < \phi_{t_i}^{r_j}$ then replace $max\phi$ with $\phi_{t_i}^{r_j}$;
calculate $S_{t_i}^{r_j}$ by Eq. (26);
if $S_{t_i}^{r_j} < 1$ **then**
  calculate $\phi_{t_i}^{r_j, f_s}$ by Eq. (29) based on Eq. (28);
end
if $max\phi < \phi_{t_i}^{r_j, f_s}$ then replace $max\phi$ with $\phi_{t_i}^{r_j, f_s}$;
return $max\phi$;

4.6 Time complexity analysis

For a workflow DAG illustrated in Sect. 3.2 where n indicates the number of DAG's tasks, the maximum number of tasks relations would be $\frac{(n(n-1))}{2}$. Calculation of EST, LFT, execution, and transfer time of resources can be obtained with the time complexity of $O(n^2)$. Categorizing tasks into levels needs processing all tasks and their precedence constraints. Therefore, it leads to $O(n^2)$ complexity at most. Deadline and budget distribution time complexity are $O(3n)$. Resource provisioning needs calculation of Eq. (29) for each resource. The maximum provisioned resource number is n. Therefore, resource provisioning needs the time complexity of $O(n^2)$ at most. Hence, the time complexity of both proposed algorithms is in the order of $O(n^2)$.

Safari's algorithm considers time and energy that leads to the time complexity of $O(n^2)$ for task ordering and $O(n^2 + n.g)$ for resource allocation with insertion policy; g denotes resource gap slots. The BDHEFT algorithm that considers time and cost as two objectives has the time complexity of $O(n^2)$.

Table 2 Experimental resource configurations based on AWS

Resource Type	ECU	Ram(GB)	Price/Hour	vCPU
m5.large	10	8	$0.096	2
m5.xlarge	16	16	$0.192	4
m5.2xlarge	37	32	$0.384	8
m5.4xlarge	70	64	$0.768	16
m5.8xlarge	128	128	$1.536	32
m5.12xlarge	168	192	$2.304	48

5 Evaluation

In this section, evaluation metrics and experimental parameters are defined to illustrate the performance of both proposed algorithms. All algorithms involved in this paper are implemented with Java programming language in Windows 10 with Intel Core i7 3630 QM processor that is equipped with 8 GB of RAM.

The proposed method of this paper can be deployed using all available Workflow Management Systems (WMS) such as Pegasus [27] and ASKALON [28]. These WMSs can manage, execute, and visualize the progress of executing workflows on Grids and Clouds. They also provide user-defined QoS in market-oriented clouds for workflow execution in more lightweight utilizing different micro-service technologies [29].

Although WMSs allow the implementation of custom scheduling algorithms, they do not have direct access to lower-level control parameters to take advantage of the DVFS technology. Therefore, researchers generally evaluate their energy-aware scheduling algorithms using simulators [4,5,15,16]. Similar to [4,5,16], we also developed a workflow scheduler based on Workflowsim to evaluate the proposed algorithms in an energy-aware fashion while considering workflows' budget and deadline constraints. Workflowsim is an extension to Cloudsim, which does not originally support execution of workflows.

The workflow applications used in this paper are synthetic workflows from the Pegasus project [30] used in Workflowsim [31]. In this paper's experimental results, λ is considered $\frac{2}{3}$ and ω is considered $\frac{1}{2}$ for Eq. (29). The BI of resources is considered hourly-based.

5.1 Experiment metrics

This section covers the performance evaluation of the proposed BDCE and BDD algorithms and compares them with state-of-the-art algorithms of ERES [15], Safari's algorithm [16], BDHEFT [14], BDSD [5], and DBCS [4]. Table 2 illustrates a data center from AWS that is simulated as a case study. The hypervisor manages the performance states (P-States) of each CPU. P-State denotes the voltage and frequency combination. Table 3 illustrates different P-States that are used for resource processors. P-States are modeled based on real Intel Xeon processors [1]. Inspired by other works, the bandwidth is considered 20 Mega-bytes per second [4,5,14].

Table 3 P-State of resource's processor based on [1]

P-State	Voltage	Frequency (GHz)
P0	1.35	3.0
P1	1.23	2.67
P2	1.05	2.27
P3	0.92	2.00
P4	0.86	1.87
P5	0.85	1.74

Eq. (30) and Eq. (31) define two variables for configuring different budget and deadline constraints. Critical path tasks (CT) denote the tasks in the longest path between the first and last task of the workflow.

$$minTime_{CT} = \sum_{t_i \in CT} (CP(t_i)) \tag{30}$$

$$maxTime_{CT} = \sum_{t_i \in CT} \left(EX_{t_i}^{\max} + \max_{t_p \in prd(t_i)} \left\{ TF_{t_p,t_i} \right\} \right) \tag{31}$$

For evaluation purposes, various constraints for the deadline and budget are initialized by Eq. (32) and Eq. (33), respectively [4,5]. Series of variations for deadline and budget constraints are acquired by exchanging the values of f_β and f_δ. $maxCost$ and $minCost$, respectively, indicate the workflow's monetary cost for execution on the most and least expensive resource type. $maxTime$ and $minTime$, respectively, indicate the workflow's scheduling length in the slowest and fastest resource. Series of variations for f_δ and f_β are considered from [0.1, 0.6]. Notably, GDL is a constant in different ranges in the results for various deadline constraints. GBD varied in the range of [0.1, 0.6] for various budget constraints.

$$GBD = minCost + f_\beta.(maxCost - minCost) \tag{32}$$

$$GDL = minTime_{CT} + f_\delta.(maxTime_{CT} - minTime_{CT}) \tag{33}$$

Eq. (34) illustrates the Success Rate (SR), which is defined as the percentage of meeting constraints like budget, deadline, and budget-deadline constraints. $|EXP_{success}|$ reflects the number of successful experiments and $|EXP_{total}|$ indicates the total number of experiments.

$$SR = \frac{|EXP_{success}| \cdot 100}{|EXP_{total}|} \tag{34}$$

Eq. (35) defines the cost ratio metric that is calculated by the monetary cost of the schedule plan to the given budget. Eq. (36) defines the time ratio metric that is

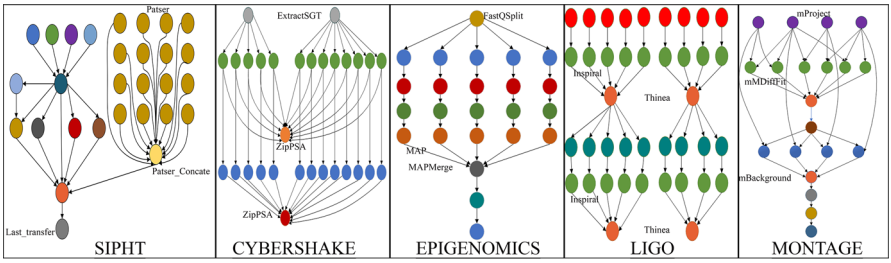


Fig. 1 Scientific applications' DAG based on [30]

calculated by Eq. (9).

$$CostRatio = \frac{WF_{cost}}{GBD} \tag{35}$$

$$TimeRatio = \frac{SL}{GDL} \tag{36}$$

REX_{r_j} calculates the busy time of resource r_j in Eq. (37). Eq. (38) is defined to analyze the total resource utilization of each algorithm. The higher utilization rate usually leads to lower energy consumption and monetary cost.

$$REX_{r_j} = \sum_{t_i \in r_j} EX_{t_i}^{r_j} \tag{37}$$

$$UR_{r_j} = \frac{REX_{r_j}}{FT(r_j) - ST(r_j)} \tag{38}$$

6 Results and discussion

This section contains the experimental results. Results of time and cost ratios with various budget and deadline constraints are represented by box plots. In these evaluation metrics, values higher than 1 for cost ratio or time ratio indicate that budget or deadline constraints are exceeded, respectively. Results were obtained by testing each workflow application with 100 tasks. In the following, the performance of the algorithms will be evaluated in five synthetic workflow applications. Tables 4, 5, and 6 illustrate the success rate of algorithms with the deadline, budget, and budget-deadline constraints, respectively. ERES and Safari's algorithms are deadline-constrained algorithms. Therefore, their success rates for deadline constraints are not inserted in Table 4. Table 7 illustrates the energy-saving ratio of the proposed algorithms with two energy-efficient algorithms proposed in [16,22]. In table 7, the energy consumption of DBCS [4] is considered as the base algorithm to evaluate the energy-saving ratio of other algorithms.

6.1 LIGO workflow

The experimental analysis for the LIGO application is shown in Fig. 2. The DBCS algorithm obtains the most affordable cost ratios in various deadlines, shown in Fig. 2a. Figures 2b–d illustrate that the proposed BDD algorithm outperforms others in a tight budget and deadline constraints. Although the results of Fig. 2c shows that the BDHEFT algorithm leads to the lowest time ratio, its success rate is not the most promising due to its higher cost ratio, as shown in Fig. 2a–d.

Figure 2e evaluates the energy consumption of all algorithms with budget and deadline constraints. It shows that the proposed BDD algorithm has the lowest energy consumption because it uses the DVFS technique. Therefore, based on Fig. 2a–c, it keeps the cost and time of the schedule plan close to the budget and deadline while improving the energy-saving as much as both constraints allow. Safari's algorithm is the second energy-efficient algorithm. The proposed BDCE algorithm has acceptable energy consumption compared to others.

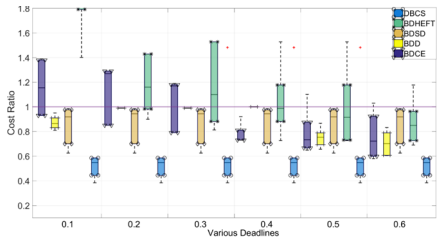
6.1.1 CYBERSHAKE workflow

Figure 3 shows the results of the CYBERSHAKE workflow application. The median value of the cost ratio in Fig. 3a shows that BDHEFT and DBCS algorithms lead to more affordable schedule plans in various deadlines. However, Fig. 3b shows that they could not outperform other algorithms in success rate; that is, they could meet the deadline but could not meet the budget. Fig. 3b exhibits that the proposed BDCE and BDD algorithms outperform others in tight deadline constraints. BDCE performs better than BDD in the tightest deadline constraint. Figure 3e illustrates the energy consumption of all algorithms for the CYBERSHAKE workflow applications. Safari's algorithm, which is a critical path-based algorithm, has the most energy-efficient performance for CYBERSHAKE. Due to the structure of CYBERSHAKE workflow in Fig. 1, Safari's algorithm could divide tasks into normal size critical paths, and that in turn led to more efficient schedule plans. The proposed BDD and BDCE algorithms have the next energy-efficient schedule plan.

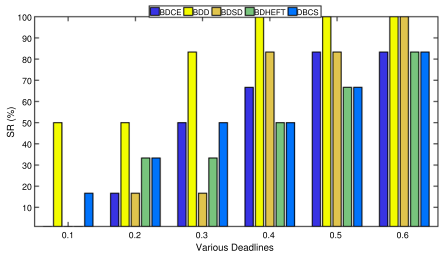
6.1.2 SIPHT workflow

Figure 4 shows the experimental result for the SIPHT application. Figure 4a shows that the DBCS algorithm has the minimum cost ratio among the algorithms. In terms of success rate, the BDD algorithm outperforms the others based on Figs. 4a-b. The BDCE algorithm is a budget-conservative algorithm and has a lower cost ratio than the BDD algorithm. BDHEFT leads to the fastest schedule plan as shown in Fig. 4c, and DBCS leads to the most affordable schedule plan based on Fig. 4a. Figure 4e demonstrates that BDD and Safari's algorithm leads to the most energy-efficient schedule plan. Overall, in terms of success rate and energy-saving ratio, BDD outperforms other algorithms.

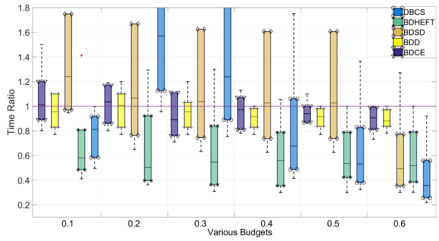
Fig. 2 Results of the LIGO workflow application



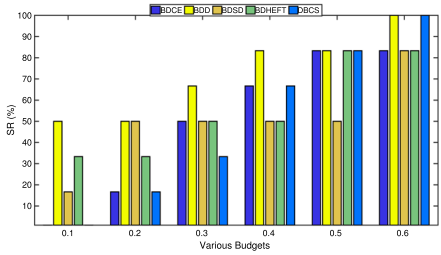
(a) Cost ratios in various given deadlines



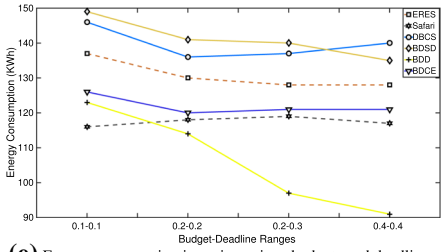
(b) The success rates in various given deadlines



(c) Time ratios in various given budgets

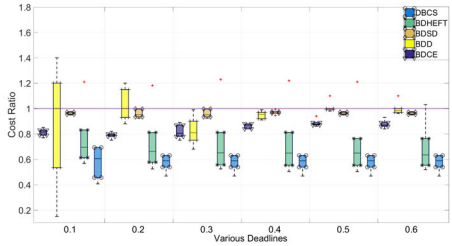


(d) The success rates in various given budgets

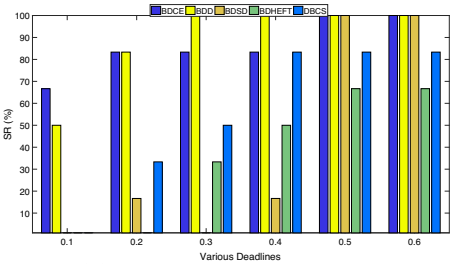


(e) Energy consumption in various given budgets and deadlines

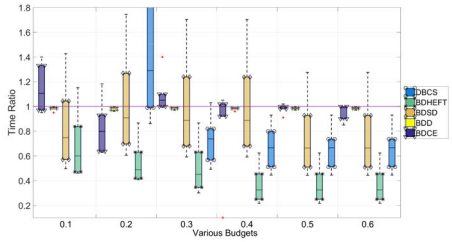
Fig. 3 Results of the CYBERSHAKE workflow application



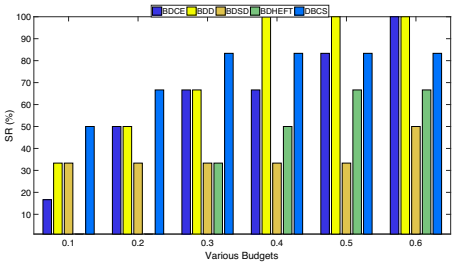
(a) Cost ratios in various given deadlines



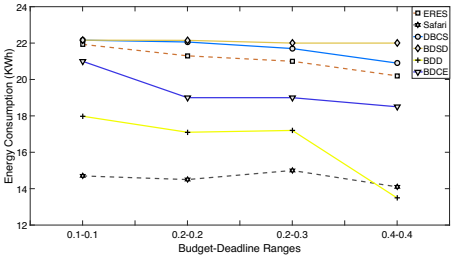
(b) The success rates in various given deadlines



(c) Time ratios in various given budgets

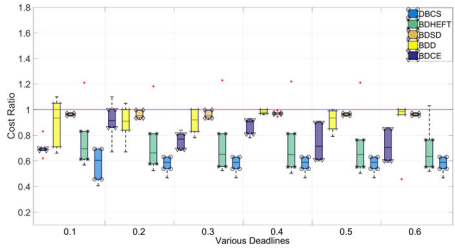


(d) The success rates in various given budgets

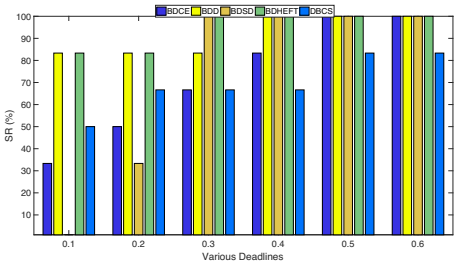


(e) Energy consumption in various given budgets and deadlines

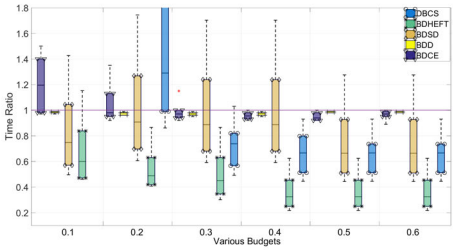
Fig. 4 Results of the SIPHT workflow application



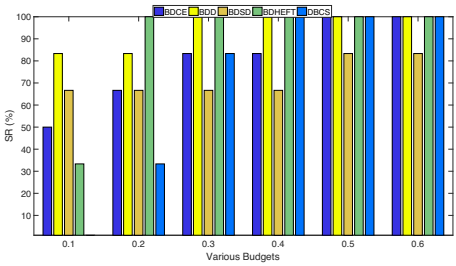
(a) Cost ratios in various given deadlines



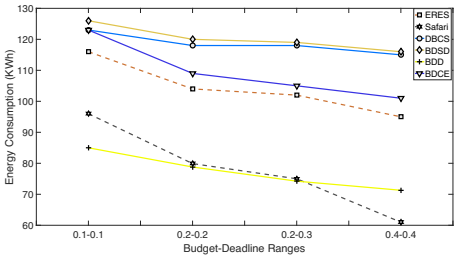
(b) The success rate in various given deadlines



(c) Time ratios in various given budgets



(d) The success rate in various given budgets



(e) Energy consumption in various given budgets and deadlines

Table 4 Average success rate for meeting the deadline constraint

Algorithms	MONTAGE	EPIGENOMICS	SIPHT	CYBERSHAKE	LIGO
BDCE	81%	88%	75%	87%	65%
BDD	83%	100%	92%	91%	94%
BDSD	59%	58%	70%	44%	49%
DBCS	62%	65%	69%	64%	47%

6.1.3 MONTAGE workflow

MONTAGE is a communication-intensive application [30]. As its structure is represented in Fig. 1, the first and second rows of the workflow contain mProject and MDiffFit types of tasks that are the most time-consuming parts of the workflow. Using high-performance resources in this type is an advantageous approach to obtain an effective schedule plan. Table 6 shows that the BDCE algorithm achieves the highest success rate for the MONTAGE application. Based on table 4, the BDD algorithm has the best success rate for deadline constraints in the MONTAGE application. Table 5 demonstrates that BDCE can meet 92% of its budget constraints. Table 7 illustrates that Safari's algorithm obtains the most energy-efficient schedule plan for MONTAGE. Because Safari's algorithm is deadline-constrained, it does not consider monetary cost as QoS, and thus it leads to a less affordable schedule plan. Table 5 shows the success rate of Safari's algorithm for meeting budget constraints. Consequently, the BDD algorithm outperforms other compared algorithms by obtaining the highest success rate for both budget and deadline constraints, shown in table 6. Table 7 demonstrates that the BDD algorithm leads to an advantageous energy-efficient schedule plan.

6.1.4 EPIGENOMICS workflow

EPIGENOMICS is a CPU-intensive application. Map type tasks in the fifth row of the EPIGENOMICS are the most time-consuming tasks [30]. This type is located in the last rows of its structure. Therefore, a budget-conservative schedule plan can save its budget by provisioning high-performance resources of this type that demands expensive monetary costs. The BDCE algorithm utilizes a budget-conservative distribution method. Hence, it economizes the cost of the schedule before the fifth row to keep enough budget for the execution of the Map tasks. Therefore, it outperforms other algorithms in achieving the greatest success rate in Tables 4, 5, and 6. Based on table 7, the proposed BDD algorithm also leads to an energy-efficient schedule plan due to utilizing the DVFS-enabled technique.

6.1.5 Utilization rate

The Utilization Rate (UR) significantly affects the QoS of the scheduling algorithms. Hence, an effective utilization rate induces an affordable cost, time, and energy-efficient schedule plan. Figure 5 represents the utilization rate of the evaluated workflow scheduling algorithms for five synthetic applications. The Result represents

Table 5 Average success rate for meeting the budget constraint

Algorithms	MONTAGE	EPIGENOMICS	SIPHT	CYBERSHAKE	LIGO
BDCE	92%	72%	96%	73%	48%
BDD	76%	64%	88%	79%	75%
BDSD	88%	61%	82%	83%	84%
DBCS	84%	74%	100%	100%	89%
Safari	33%	16%	16%	50%	0%
ERES	66%	75%	94%	100%	50%

Table 6 Average success rate for simultaneous budget and deadline constraints

Algorithms	MONTAGE	EPIGENOMICS	SIPHT	CYBERSHAKE	LIGO
BDCE	81%	72%	75%	73%	48%
BDD	76%	64%	88%	79%	75%
BDSD	59%	58%	70%	44%	49%
DBCS	62%	65%	69%	64%	47%

Table 7 Average energy-saving ratio

Algorithms	MONTAGE	EPIGENOMICS	SIPHT	CYBERSHAKE	LIGO
BDCE	18%	9%	13%	23%	11%
BDD	43%	39%	45%	41%	34%
Safari	54%	24%	36%	49%	26%
ERES	25%	15%	11%	18%	9%

algorithms' UR for different workflows. The proposed BDD algorithm utilizes the DVFS technique to provide an energy-efficient schedule plan. Therefore, it can reduce the frequency of processors when there is slack time for a task. Hence, its resource utilization rate is higher than other algorithms. Safari's algorithm does not consider cost and utilization rate in the schedule plan. The DVFS technique can facilitate the utilization rate if the BI of resources is considered. Because Safari's algorithm considers the slack time of the tasks based on the deadline (not the idle time of resources), it outperforms other algorithms in terms of resource utilization rate. The proposed BDCE algorithm has a competitive utilization rate based on Fig. 5.

7 Conclusion

This paper proposes two energy-efficient workflow scheduling algorithms for simultaneous budget and deadline constraints. The proposed algorithms are suitable for workflow applications running in cloud environments. Various metrics such as time and cost ratios, the success rate, energy consumption, energy-saving ratio, and utilization rate are employed to evaluate workflow scheduling algorithms. Results indicate that the BDCE leads to an affordable cost and efficient energy policy for budget-deadline constrained workflow applications in non-DVFS resources. The BDD algorithm leads

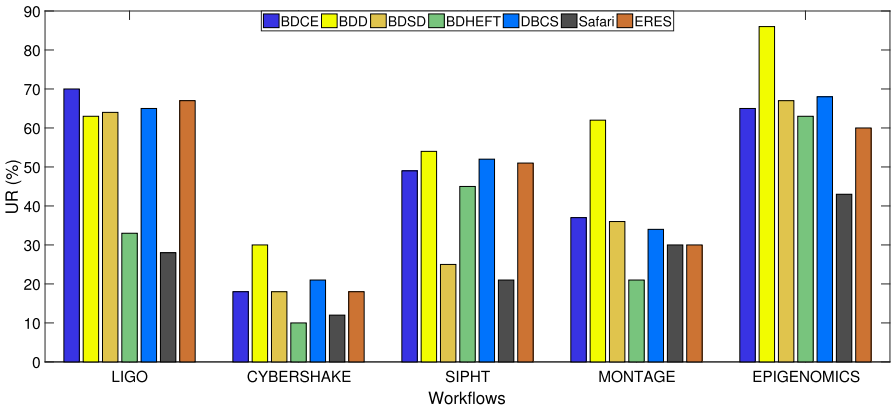


Fig. 5 Algorithms' UR in various workflow applications

to a more eminent success rate and energy efficiency for budget-deadline constrained workflows in the DVFS-enabled resources. Experiments illustrate that BDCE and BDD algorithms outperform other algorithms on various fronts. The proposed algorithms are suitable for cloud environments that are sensitive to the budget and deadline. The BDD algorithm is appropriate for DVFS-enabled resources to reach a more energy-efficient schedule plan. The BDCE algorithm is suitable for non-DVFS resources with a budget-conservative schedule plan. We will investigate the scheduling methods for reliability as QoS and dynamic multi-workload scheduling in future work.

References

1. Stavrinides GL, Karatza HD (2019a) An energy-efficient, QoS-aware and cost-effective scheduling approach for real-time workflow applications in cloud computing systems utilizing DVFS and approximate computations. Future Generat Comp Syst 96:216–226. https://doi.org/10.1016/j.future.2019.02.019
2. Deelman E, Vahi K, Juve G, Rynge M, Callaghan S, Maechling PJ, Mayani R, Chen W, Ferreira Da Silva R, Livny M, Wenger K (2015) Pegasus, a workflow management system for science automation. Future Generat Comput Syst 46:17–35. https://doi.org/10.1016/j.future.2014.10.008
3. Sharifi M, Shahrivari S, Salimi H (2013) PASTA: a power-aware solution to scheduling of precedence-constrained tasks on heterogeneous computing resources. Computing 95(1):67–88. https://doi.org/10.1007/s00607-012-0212-1
4. Arabnejad H, Barbosa JG, Prodan R (2016a) Low-time complexity budget-deadline constrained workflow scheduling on heterogeneous resources. Future Generat Comp Syst 55:29–40. https://doi.org/10.1016/j.future.2015.07.021
5. Sun T, Xiao C, Xu X (2019) A scheduling algorithm using sub-deadline for workflow applications under budget and deadline constrained. Cluster Comput 22(3):5987–5996. https://doi.org/10.1007/s10586-018-1751-9
6. Xie G, Zeng G, Li R, Li K (2019) Scheduling Parallel Applications on Heterogeneous Distributed Systems. Springer, Singapore. https://doi.org/10.1007/978-981-13-6557-7
7. Abrishami S, Naghibzadeh M, Epema DHJ (2013) Deadline-constrained workflow scheduling algorithms for infrastructure as a service Clouds. Future Generat Comput Syst 29(1):158–169. https://doi.org/10.1016/j.future.2012.05.004
8. Singh V, Gupta I, Jana PK (2019) An energy efficient algorithm for workflow scheduling in IaaS Cloud. J Grid Comput. https://doi.org/10.1007/s10723-019-09490-2

9. Panda SK, Jana PK (2015) Efficient task scheduling algorithms for heterogeneous multi-cloud environment. J Supercomput 71(4):1505–1533. https://doi.org/10.1007/s11227-014-1376-6
10. Zheng W, Qin Y, Bugingo E, Zhang D, Chen J (2018) Cost optimization for deadline-aware scheduling of big-data processing jobs on clouds. Future Generat Comput Syst 82:244–255. https://doi.org/10.1016/j.future.2017.12.004
11. Topcuoglu H, Hariri S, Min-You Wu (2002) Performance-effective and low-complexity task scheduling for heterogeneous computing. IEEE Trans Parallel Distrib Syst 13(3):260–274. https://doi.org/10.1109/71.993206
12. Zheng W, Sakellariou R (2013) Budget-deadline constrained workflow planning for admission control. J Grid Comput 11(4):633–651. https://doi.org/10.1007/s10723-013-9257-4
13. Ullman JD (1975) NP-complete scheduling problems. J Comput Syst Sci 10(3):384–393
14. Verma A, Kaushal S (2015) Cost-time efficient scheduling plan for executing workflows in the Cloud. J Grid Comput 13(4):495–506. https://doi.org/10.1007/s10723-015-9344-9
15. Garg N, Singh D, Goraya MS (2020) Energy and resource efficient workflow scheduling in a virtualized cloud environment. Cluster Comput 4:1–31. https://doi.org/10.1007/s10586-020-03149-4
16. Safari M, Khorsand R (2018) Energy-aware scheduling algorithm for time-constrained workflow tasks in DVFS-enabled cloud environment. Simul Modell Pract Theory 87(July):311–326. https://doi.org/10.1016/j.simpat.2018.07.006
17. Casas I, Taheri J, Ranjan R, Wang L, Zomaya AY (2018) GA-ETI: an enhanced genetic algorithm for the scheduling of scientific workflows in cloud environments. J Comput Sci 26:318–331. https://doi.org/10.1016/j.jocs.2016.08.007
18. Casas I, Taheri J, Ranjan R, Wang L, Zomaya AY (2017) A balanced scheduler with data reuse and replication for scientific workflows in cloud computing systems. Future Generat Comput Syst 74:168–178. https://doi.org/10.1016/j.future.2015.12.005
19. Arabnejad V, Bubendorfer K, Ng B (2016b) Budget Distribution Strategies for Scientific Workflow Scheduling in Commercial Clouds. In: IEEE 12th International Conference on e-Science Budget, IEEE, pp 137–146
20. Stavrinides GL, Karatza HD (2019b) An energy-efficient, QoS-aware and cost-effective scheduling approach for real-time workflow applications in cloud computing systems utilizing DVFS and approximate computations. Future Generat Comput Syst 96:216–226. https://doi.org/10.1016/j.future.2019.02.019
21. Li Z, Ge J, Hu H, Song W, Hu H, Luo B (2018) Cost and energy aware scheduling algorithm for scientific workflows with deadline constraint in clouds. IEEE Trans Serv Comput 11(4):713–726
22. Tang Z, Qi L, Cheng Z, Li KKK, Khan SU, Li KKK (2016) An energy-efficient task scheduling algorithm in DVFS-enabled cloud environment. J Grid Comput 14(1):55–74. https://doi.org/10.1007/s10723-015-9334-y
23. Rizvandi NB, Taheri J, Zomaya AY, Lee YC (2010) Linear Combinations of DVFS-Enabled Processor Frequencies to Modify the Energy-Aware Scheduling Algorithms. In: 2010 10th IEEE/ACM International Conference on Cluster, Cloud and Grid Computing, pp 388–397, https://doi.org/10.1109/CCGRID.2010.38
24. Rizvandi NB, Taheri J, Zomaya AY (2011) Some observations on optimal frequency selection in DVFS-based energy consumption minimization. J Parallel Distrib Comput 71(8):1154–1164. https://doi.org/10.1016/j.jpdc.2011.01.004
25. Pham TP, Durillo JJ, Fahringer T (2017) Predicting workflow task execution time in the cloud using a two-stage machine learning approach. IEEE Trans Cloud Comput 99(1):1–1. https://doi.org/10.1109/TCC.2017.2732344
26. Yuan Y, Li X, Wang Q, Zhang Y (2008) Bottom level based heuristic for workflow scheduling in grids. Chin J Comput Chin Edit 31(2):282
27. Deelman E, Singh G, Su MH, Blythe J, Gil Y, Kesselman C, Mehta G, Vahi K, Berriman GB, Good J, Others, (2005) Pegasus: a framework for mapping complex scientific workflows onto distributed systems. Sci Programm 13(3):219–237
28. Wieczorek M, Prodan R, Fahringer T (2005) Scheduling of scientific workflows in the ASKALON grid environment. Acm Sigmod Record 34(3):56–62
29. Hilman MH, Rodriguez MA, Buyya R (2020) Multiple workflows scheduling in multi-tenant distributed systems: a taxonomy and future directions. ACM Comput Surv. https://doi.org/10.1145/3368036

30. Juve G, Chervenak A, Deelman E, Bharathi S, Mehta G, Vahi K (2013) Characterizing and profiling scientific workflows. Future Generat Comput Syst 29:682–692. https://doi.org/10.1016/j.future.2012.08.015
31. Chen W, Rey M, Rey M (2012) WorkflowSim : A Toolkit for Simulating Scientific Workflows in Distributed Environments. In: The 8th IEEE International Conference on eScience 2012 (eScience 2012) pp 1–8, https://doi.org/10.1109/eScience.2012.6404430

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.